Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2025-26

Class:	**TE**	**Semester:**	**VI**
Course Code:	**CSL604**	**Course Name:**	**Machine Learning Lab**

Name of Student:	**Shravani Sandeep Raut**
Roll No. :	**40**
Experiment No.:	**06**
Title of the Experiment:	**To learn how the Expectation–Maximization (EM) algorithm finds patterns or groups in data when some information is hidden or unknown.**
Date of Performance:	**12/02/2026**
Date of Submission:	**26/02/2026**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty : Mrs. Sneha Yadav

Signature :

D

Aim: To learn how the **Expectation–Maximization (EM) algorithm** finds patterns or groups in data when some information is **hidden or unknown**.

Objective: To understand the basic idea of EM.
To learn how EM works in two simple steps.
To see how EM is used for clustering data.

Theory:

What is the EM Algorithm? (In Simple Words)
The EM algorithm is used when:

- We have data
- But we do not know everything about it (some values are hidden)

EM helps us guess the missing information, improve the guess step by step, and finally get the best result.

Example:

Imagine a class where students are mixed, and teacher don't know which student belongs to **Group A** or **Group B**.

Teacher:

- **Guess** the groups
- **Check** how good your guess is
- **Improve** the grouping
- Repeat until grouping becomes stable This is exactly how EM works.

- **Working of EM Algorithm**

EM works in **two repeating steps**:

1 **Expectation Step (E-Step)**

- The algorithm **estimates** which group each data point belongs to
- It calculates **probabilities** instead of making a hard decision

Example:

"This point belongs to Group A with 70% chance and Group B with 30% chance."

Maximization Step (M-Step)

- The algorithm **updates the model parameters** (mean, variance, etc.)
- Uses the probabilities from the E-step to improve the grouping*Example:*

Recalculate the center of each group based on current members.

Repeat

- E-Step → M-Step → E-Step → M-Step
- Continue until there is **no major change**

- **Simple Flow of EM Algorithm**

Start
↓
Initialize values
↓
Expectation Step (Guess group probabilities)
↓
Maximization Step (Update parameters)
↓
Repeat until stable
↓
Stop

- **Simple Implementation Idea (No Math)**
  - Start with **random group centers**
  - Assign data points **probabilistically**
  - Update group centers
  - Repeat until results stop changing

- **Where EM is Used?**

- **Clustering (Gaussian Mixture Model)**
- **Image segmentation**
- **Speech recognition**
- **Handling missing data**
- **Machine learning & AI**

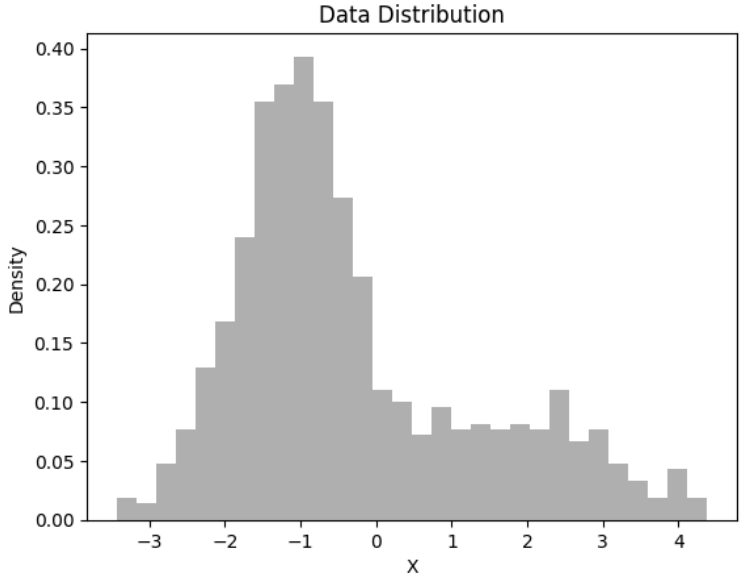
o **Advantages (Simple)**

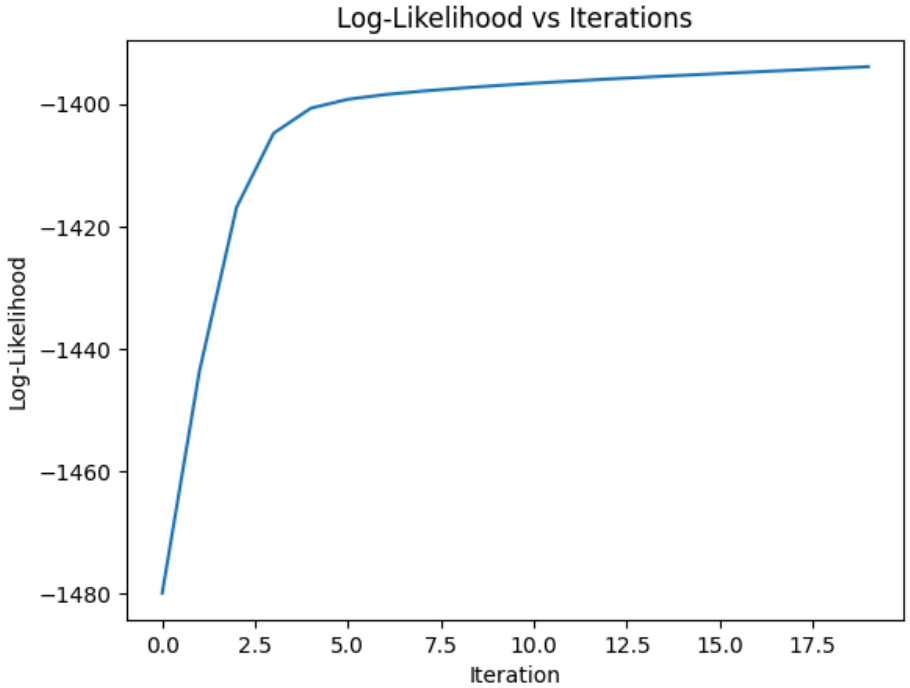
- Works well even if data is **incomplete**
- Easy concept
- Improves results step by step

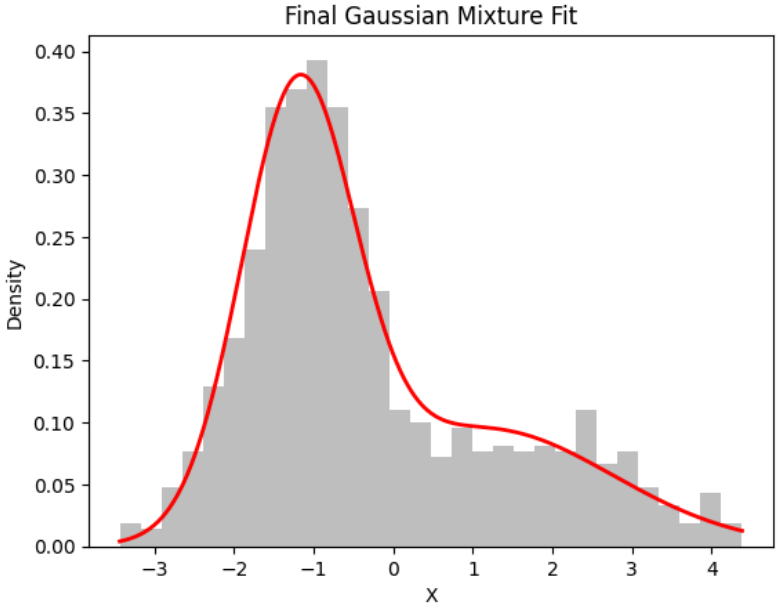
Implementation:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

np.random.seed(0)

# Generate sample data (two Gaussians)
X = np.concatenate([
    np.random.normal(2, 1, 200),
    np.random.normal(-1, 0.8, 600)
])

# Plot initial data histogram
plt.hist(X, bins=30, density=True, alpha=0.6, color='gray')
plt.title("Data Distribution")
plt.xlabel("X")
plt.ylabel("Density")
plt.show()

# Initialize parameters
mu1, mu2 = np.random.choice(X, 2)
sigma1, sigma2 = 1, 1
pi1, pi2 = 0.5, 0.5

log_likelihoods = []

# EM Algorithm
for _ in range(20):

    # E-step
    r1 = pi1 * norm.pdf(X, mu1, sigma1)
```

```
    r2 = pi2 * norm.pdf(X, mu2, sigma2)
    total = r1 + r2
    r1 /= total
    r2 /= total

    # M-step
    mu1 = np.sum(r1 * X) / np.sum(r1)
    mu2 = np.sum(r2 * X) / np.sum(r2)

    sigma1 = np.sqrt(np.sum(r1 * (X - mu1)**2) / np.sum(r1))
    sigma2 = np.sqrt(np.sum(r2 * (X - mu2)**2) / np.sum(r2))

    pi1 = np.mean(r1)
    pi2 = np.mean(r2)

    # Log-likelihood
    ll = np.sum(np.log(
        pi1 * norm.pdf(X, mu1, sigma1) +
        pi2 * norm.pdf(X, mu2, sigma2)
    ))
    log_likelihoods.append(ll)

# Plot log-likelihood vs iterations
plt.plot(log_likelihoods)
plt.title("Log-Likelihood vs Iterations")
plt.xlabel("Iteration")
plt.ylabel("Log-Likelihood")
plt.show()

# Plot final fitted mixture
x_vals = np.linspace(min(X), max(X), 1000)
mixture = (pi1 * norm.pdf(x_vals, mu1, sigma1) +
           pi2 * norm.pdf(x_vals, mu2, sigma2))

plt.hist(X, bins=30, density=True, alpha=0.5, color='gray')
plt.plot(x_vals, mixture, color='red', linewidth=2)
plt.title("Final Gaussian Mixture Fit")
plt.xlabel("X")
plt.ylabel("Density")
plt.show()

# Print final parameters
print("Component 1:", mu1, sigma1, pi1)
print("Component 2:", mu2, sigma2, pi2)
```


Data Distribution
Density
X
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
−3
−2
−1
0
1
2
3
4


Log-Likelihood vs Iterations
Log-Likelihood
Iteration
−1400
−1420
−1440
−1460
−1480
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5



Conclusion:

The Expectation-Maximization (EM) algorithm serves as a powerful iterative method for uncovering hidden structures within data by alternating between an E-step, which assigns data points to latent clusters, and an M-step, which refines the statistical parameters of those clusters. In this implementation, the algorithm successfully reconstructs a complex distribution by decomposing it into two distinct Gaussian components, effectively converging on the optimal means and variances that best explain the observed dataset.